35

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

엔드투엔드 테스트

제2부 NestJS 핵심 · 8장 테스트 기초

고객이 지나가는 길을 처음부터 끝까지



© 한상훈 NestJS 백엔드 마스터

명령을 못 찾는 e2e

터미널

```
$ npm run test:e2e

> ticketing@0.0.1 test:e2e
> viapps/ticketing/test run --config ./vitest.config.e2e.ts
```

→ 출력: "'viapps'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는 배치 파일이 아닙니다."

→ 서비스를 둘로 나눌 때 CLI가 경로를 바꾸며 `vitest` 안의 `test`까지 치환

→ 그 뒤로 e2e 테스트 실행 기록 없음

→ 빨간불은 눈에 띄지만, 안 도는 테스트는 아무 소리 없음 → 명령 실패도 실패로

© 한상훈 NestJS 백엔드 마스터

고치고 돌리면

package.json

```
-    "test:e2e": "viapps/ticketing/test run --config ./vitest.config.e2e.ts"
+    "test:e2e": "vitest run --config ./vitest.config.e2e.ts"
```

○ ○ ○ 터미널

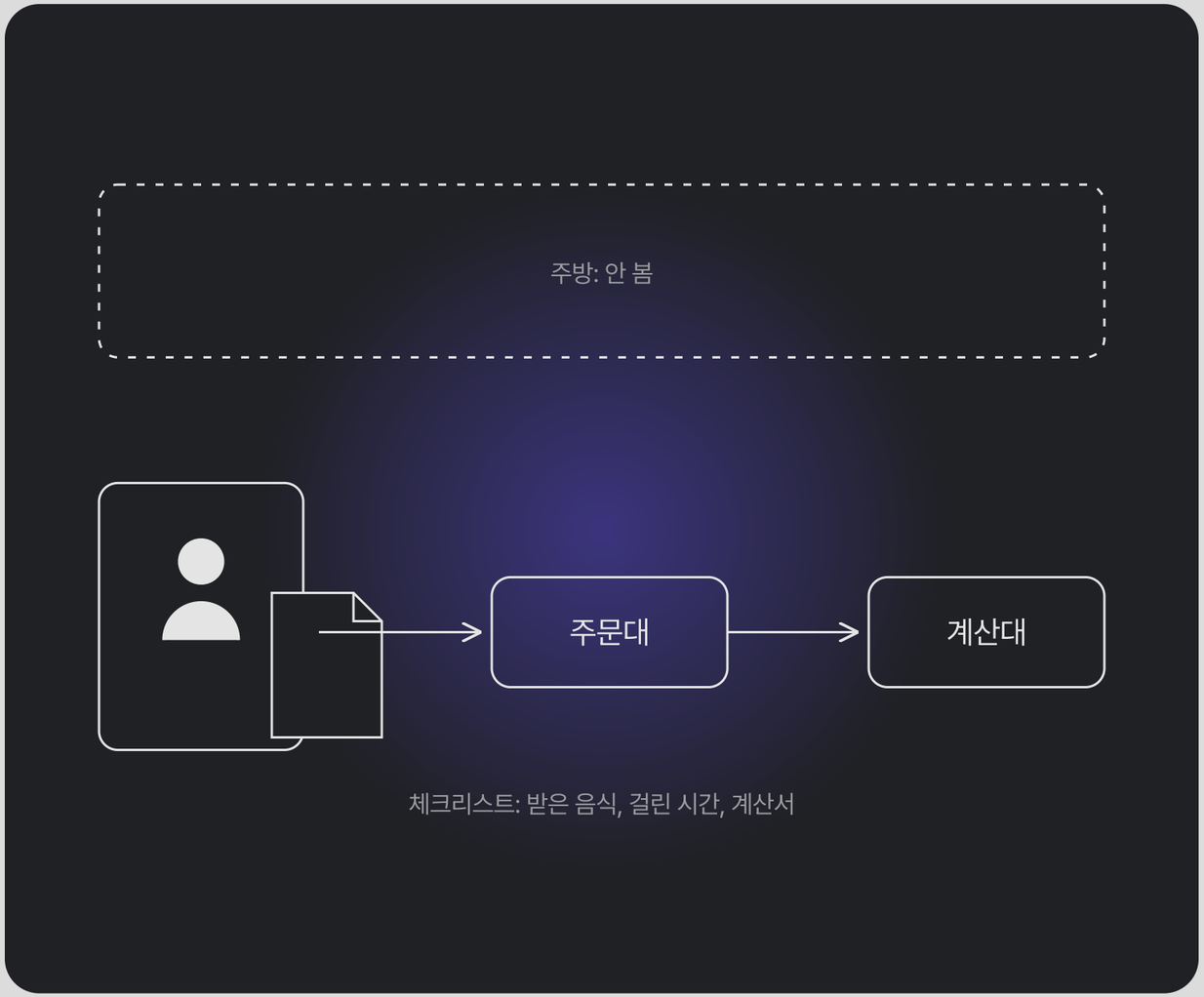
```
 × apps/notifier/test/app.e2e-spec.ts > NotifierController (e2e) > / (GET) 355ms
   → expected 'Hello World!' response body, got '알림: A구역 3열 15번 예매 확정'
결제 설정: {"mode":"http","timeoutMs":3000}
 × apps/ticketing/test/app.e2e-spec.ts > AppController (e2e) > / (GET) 45ms
   → Failed to parse URL from undefined/token
   → Cannot read properties of undefined (reading 'close')
 Test Files  2 failed (2)
```

→ 알림 워커: 낡은 기댓값

→ 예매 서버: 결제 모듈이 결제사 주소 없이 토큰 요청, 그 뒤 `afterEach`의 `close`도 실패

© 한상훈 NestJS 백엔드 마스터

미스터리 쇼퍼



→ 고객인 척 들어와 주문부터 계산까지 한 번에 경험

→ 주방 안은 안 봄: 받은 음식, 걸린 시간, 계산서만 확인

→ 엔드투엔드 테스트: HTTP 요청으로 들어가 응답만 확인

© 한상훈 NestJS 백엔드 마스터

엔드투엔드 테스트

→ 엔드투엔드 테스트(end-to-end test, e2e): 서버 전체를 띄우고 HTTP 요청으로만 들어가서 응답만 확인하는 테스트

→ 지나가는 것: 라우팅, 파이프, 가드, 컨트롤러, 서비스, 저장소, DB, 응답 직렬화

→ 확인하는 것: 상태 코드, 응답 본문

→ 안쪽 클래스는 직접 부르지 않음

© 한상훈 NestJS 백엔드 마스터

테스트용 Nest 앱과 supertest

```
beforeEach(async () => {
  const moduleFixture: TestingModule = await Test.createTestingModule({
    imports: [AppModule],
  })
    .overrideProvider(PaymentGateway)
    .useClass(FakePaymentGateway)
    .overrideProvider(SmsOutbox)
    .useValue({ enqueue: () => {} })
    .compile();

  app = moduleFixture.createNestApplication();
  setupApp(app);
  await app.init();
});

it('/ (GET)', () => {
  return request(app.getHttpServer())
    .get('/')
    .expect(200)
    .expect('예매 API: A구역 3열 15번');
});
```

○ ○ ○ 터미널

```
 ✓ apps/notifier/test/app.e2e-spec.ts > NotifierController
(e2e) > / (GET) 289ms
 ✓ apps/ticketing/test/app.e2e-spec.ts > AppController (e2e) >
/ (GET) 155ms
```

→ `createNestApplication()`: 테스트 모듈로 Nest 앱 인스턴스 생성, `listen` 없이 `init`만

→ supertest: 포트를 열지 않고 Node HTTP 서버에 직접 요청

→ `.send(객체)`: JSON 본문과 Content-Type, 응답은 `body`(JSON)·`text`(글자), 헤더도 `expect`로

© 한상훈 NestJS 백엔드 마스터

main.ts가 하는 일

```
process.loadEnvFile();

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(requestId, requestLog); // body parser보다 먼저
  app.use(compression());
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
  const server = app.getHttpServer();
  server.keepAliveTimeout = 65_000; // 로드 밸런서 유휴 시간(60초)보다 길게
  app.connectMicroservice<MicroserviceOptions>({
    transport: Transport.TCP,
    options: { host: '127.0.0.1', port: Number(process.env.TCP_PORT ?? 4001) },
  }, { inheritAppConfig: true });
  app.enableShutdownHooks();
  await app.init();
  await app.startAllMicroservices();
  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

→ 34강 끝 코드의 main.ts

→ e2e 테스트는 main.ts를 실행하지 않음

© 한상훈 NestJS 백엔드 마스터

e2e용 Nest 앱에 없는 것

main.ts에서 하는 일	e2e용 Nest 앱에서	결과
.env 읽기 (loadEnvFile)	안 읽음	결제사 주소 없음: Failed to parse URL
TCP 마이크로서비스 연결	연결 안 함	TCP 메시지(reservations.find)는 e2e로 확인 불가
종료 훅 켜기 (enableShutdownHooks)	안 켬	시그널 종료 동작은 다름
useGlobalPipes, app.use 미들웨어(요청 ID·로그, 압축)	안 걸림	검증이 운영과 다름 (400이 통과로)
AppModule의 APP_FILTER·APP_GUARD·APP_PIPE	걸림	모듈에 들어 있는 전역 설정은 그대로

→ 서버 설정은 함수 하나로 빼서 main.ts와 e2e 테스트가 같이 호출

→ 밖으로 나가는 것은 오버라이드

→ main.ts에 한 줄 추가할 때: e2e용 Nest 앱에도 필요하면 setupApp으로, 밖과 연결이면 테스트에서 오버라이드

© 한상훈 NestJS 백엔드 마스터

main.ts와 e2e가 같이 부르는 설정

```ts
// setup-app.ts
export function setupApp(app: INestApplication) {
  app.use(requestId, requestLog); // body parser보다 먼저
  app.use(compression());
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
}

// main.ts: NestFactory.create 다음 줄 / e2e: createNestApplication 다음 줄
setupApp(app);
```

```
예매 본문에 없는 필드 coupon을 섞은 POST /reservations
setupApp 없는 e2e용 Nest 앱  201 {"id":1,"seatId":"A-3-15","approvalId":"fake-110000",...}
setupApp 부른 e2e용 Nest 앱  400 {"type":"about:blank","title":"Bad Request","status":400,"errors":["property coupon should not exist"],...}
```

→ 400 본문이 Problem Details: APP_FILTER로 등록한 필터는 e2e용 Nest 앱에도 걸림

→ keepAliveTimeout, TCP 연결, 종료 훅: 포트를 여는 운영 서버에만 (main.ts에 남김)

© 한상훈 NestJS 백엔드 마스터

고객 흐름의 무대

```
describe('예매 흐름 (e2e)', () => {
  let container: StartedPostgreSqlContainer;
  let app: INestApplication;
  const outbox = { enqueue: vi.fn() };

  beforeAll(async () => {
    container = await new PostgreSqlContainer('postgres:17-alpine').withName('ch08-pg-35').start();
    const pool = new Pool({ connectionString: container.getConnectionUri() });
    await pool.query(readFileSync('db/schema.sql', 'utf8'));
    await pool.end();
    process.env.DATABASE_URL = container.getConnectionUri();

    const moduleRef = await Test.createTestingModule({ imports: [AppModule] })
      .overrideProvider(PaymentGateway)
      .useClass(FakePaymentGateway)
      .overrideProvider(SmsOutbox)
      .useValue(outbox)
      .compile();
    app = moduleRef.createNestApplication();
    setupApp(app);
    await app.init();
  });
```

→ DB: 진짜 PostgreSQL 컨테이너, DATABASE_URL은 compile 전에

→ 결제사와 문자 발송함: 밖으로 나가는 것만 대역

© 한상훈 NestJS 백엔드 마스터

고객 한 명의 흐름

```
it('공연 등록부터 영수증까지', async () => {
  const http = () => request(app.getHttpServer());

  await http().post('/concerts').send({ title: '가을 콘서트', venue: '올림픽홀' }).expect(201);
  await http().post('/performances').send({ concertId: 1, startsAt: '2026-12-24T19:00' }).expect(201);
  await http().post('/performances/1/seats').expect(201);

  const held = await http().post('/performances/1/seats/A-3-15/hold').expect(201);
  expect(held.body).toEqual({ id: 'A-3-15', label: 'A구역 3열 15번', status: 'HELD' });
  await http().post('/performances/1/seats/A-3-15/hold').expect(409);

  const reserved = await http()
    .post('/reservations')
    .send({ seatId: 'A-3-15', amount: 110000, buyer: '김민수' })
    .expect(201);
  expect(reserved.body.approvalId).toBe('fake-110000');
  expect(outbox.enqueue).toHaveBeenCalledWith('김민수', 'A-3-15 예매가 확정되었습니다');

  const receipt = await http().get(`/reservations/${reserved.body.id}/receipt`).expect(200);
  expect(receipt.text).toBe(`영수증 PDF: 예매 ${reserved.body.id}번`);
});
```

© 한상훈 NestJS 백엔드 마스터

실행 결과

○ ○ ○ 터미널

```
$ npm run test:e2e
 ✓ app.e2e-spec.ts > NotifierController (e2e) > / (GET) 289ms
 ✓ app.e2e-spec.ts > AppController (e2e) > / (GET) 155ms
영수증 모듈 코드 불러옴 (0.8초)
 ✓ booking.e2e-spec.ts > 예매 흐름 (e2e) > 공연 등록부터 영수증까지 1107ms
 ✓ booking.e2e-spec.ts > 예매 흐름 (e2e) > 없는 회차에는 좌석을 열 수 없다 4ms
 Test Files  3 passed (3)
      Tests  4 passed | 1 todo (5)
```

→ 흐름 테스트 1초 남짓: 대부분 영수증 모듈을 처음 불러오는 0.8초

→ 한 테스트에 이어 쓴 이유: 뒤 요청이 앞 요청 결과(회차, 좌석, 예매 번호)에 의존

→ 단언: 상태 코드 위주, 본문은 고객이 쓰는 값만

데이터 블록: e2e 실행 3 files, 4 passed, 1 todo, Duration 5.87s(컨테이너 준비 포함, 강사 PC 실측 2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

붙잡지 않은 좌석

```
  it('붙잡지 않은 좌석으로 예매하면 409', async () => {
    await request(app.getHttpServer())
      .post('/reservations')
      .send({ seatId: 'Z-9-99', amount: 110000, buyer: '이지은' })
      .expect(409);
  });
```

○ ○ ○ 터미널

```
[예매] Z-9-99 결제 완료 (unknown)
 × 예매 흐름 (e2e) > 붙잡지 않은 좌석으로 예매하면 409 4ms
   → expected 409 "Conflict", got 201 "Created"
```

→ Z-9-99: 열린 적도 없는 좌석, 결제와 예매 확정까지 진행

→ 좌석 테스트: 통과 / 예매 테스트: 통과 / 둘 사이: 연결 없음

© 한상훈 NestJS 백엔드 마스터

빈틈을 남기는 법

```
// 지금은 201: 예매가 좌석 상태를 확인하지 않는다(좌석 애그리거트에서 고친다)
it.todo('붙잡지 않은 좌석으로 예매하면 409');
```

○ ○ ○ 터미널

```
Tests  4 passed | 1 todo (5)
```

→ 고치려면: 예매 요청이 회차와 좌석을 받고, 예매가 좌석 붙잡기를 확인

→ `it.todo`: 실행은 안 하고 목록에 남김, 빨간 테스트로 두지 않음

→ 정상 흐름에 이상한 행동 섞기: 남이 붙잡은 좌석으로 예매, 같은 예매 두 번, 0원 결제

© 한상훈 NestJS 백엔드 마스터

닫지 않으면

```
  afterAll(async () => {
    // await app.close();
    await container.stop();
  });
```

○ ○ ○ 터미널

```
 ✓ 예매 흐름 (e2e) > 공연 등록부터 영수증까지 1060ms
 ✓ 예매 흐름 (e2e) > 없는 회차에는 좌석을 열 수 없다 4ms
⎯⎯⎯⎯⎯⎯ Unhandled Errors ⎯⎯⎯⎯⎯⎯
Vitest caught 1 unhandled error during the test run.
This might cause false positive tests. Resolve unhandled errors to make sure your tests are not affected.
⎯⎯⎯⎯⎯ Uncaught Exception ⎯⎯⎯⎯⎯
error: terminating connection due to administrator command
```

→ 테스트는 전부 통과, 그 뒤에 처리 안 된 에러

→ 처리 안 된 에러: 다른 테스트 도중에 터지면 멀쩡한 테스트가 실패로 보이거나 진짜 실패가 묻힘 → 초록이어도 실패로 취급

© 한상훈 NestJS 백엔드 마스터

정리 순서

→ `app.close()`: 종료 훅 실행 → `PgSeatsRepository.onModuleDestroy` → Pool 닫기

→ `container.stop()`: DB 종료

→ 순서: Nest 앱을 먼저 닫고 컨테이너를 내림

→ 순서를 바꾸면: 열린 연결이 남은 채로 DB가 내려가 연결 끊김 에러

→ 끝까지 정리해야 다음 실행과 CI가 깨끗함

→ 테스트마다 Nest 앱을 만들면 테스트마다 close (템플릿 테스트의 `afterEach`)

© 한상훈 NestJS 백엔드 마스터

명령 네 개

명령	도는 파일	필요한 것
`npm test`	`apps/*/src/**/*.spec.ts` (유닛, 모듈 조립)	Node
`npm run test:types`	테스트 파일 포함 타입 검사	Node
`npm run test:int`	`*.int-spec.ts` (통합)	Node + Docker
`npm run test:e2e`	`apps/*/test/*.e2e-spec.ts` (엔드투엔드)	Node + Docker

→ 지금 개수: 유닛·모듈 조립 19, 통합 4, 엔드투엔드 4 + 할 일 1

© 한상훈 NestJS 백엔드 마스터

이 장에서 잡은 것

테스트	잡은 것
유닛 (목)	결제가 실패해도 확정 문자를 보내는 순서
유닛 (가짜 타이머)	12월 25일부터 실패하는 날짜 의존 테스트
테스트 모듈	초기화 훅 안의 문자 발송사 호출, init 누락
통합	`count` 문자열로 안 뜨는 매진, 두 번 열기의 기본 키 위반
엔드투엔드	붙잡지 않은 좌석으로 되는 예매, 깨진 `test:e2e`

© 한상훈 NestJS 백엔드 마스터

얼마나 둘까



→ 유닛: 많이 (규칙마다, 밀리초)

→ 통합: 조금 (DB에 닿는 코드마다, 초 단위)

→ 엔드투엔드: 몇 개 (고객 흐름마다, 초 단위)

→ 거꾸로 된 모양(e2e만 많음): 실행 수십 분, 실패 원인 찾기 어려움, 아이스크림 콘

© 한상훈 NestJS 백엔드 마스터

엔드투엔드 테스트의 규칙

"엔드투엔드 테스트는 고객이 지나가는 길을 처음부터 끝까지 지나가 보고, 상태 코드와 응답만 확인합니다."